
Priority Scheduling

Operating Systems and Systems Programming Project

Section: 6

Team No.: 20

Team Members:

Somisetty Ketana Sindhu-2520030487

Yerrolaa Eekshitha— 2520030615

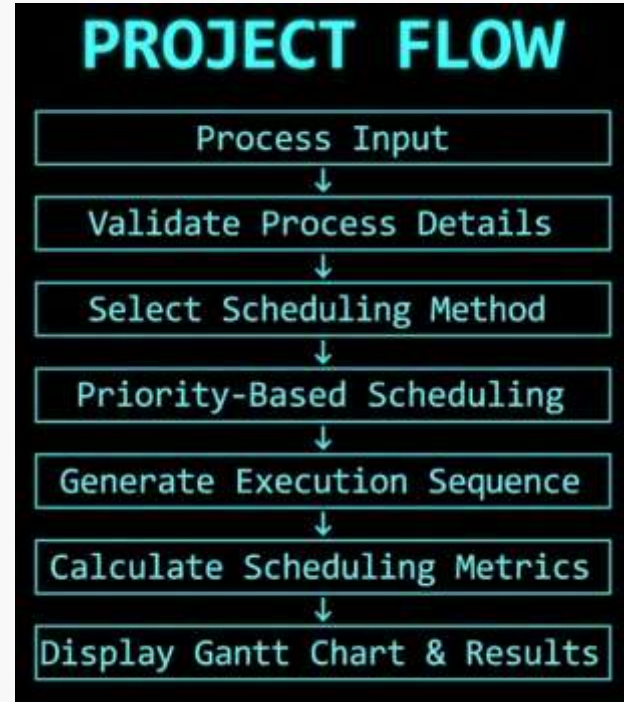
Poojitha Rani— 2520030460

Abstract

- A Priority Scheduling-based CPU scheduling simulator for Linux.
- Implements **Preemptive and Non-Preemptive Priority Scheduling**.
- Accepts **Process ID, Arrival Time, Burst Time and Priority**.
- Calculates **Completion Time, Turnaround Time, Waiting Time and Response Time**.
- Displays process execution order using a **Gantt chart**.
- Demonstrates how an Operating System allocates CPU resources based on process priorities.
- Provides a simple and practical understanding of Priority Scheduling.

Introduction

- Priority Scheduling is a CPU scheduling algorithm used by Operating Systems.
- Processes are executed based on their assigned priority.
- Provides:
 - **Preemptive Priority Scheduling**
 - **Non-Preemptive Priority Scheduling**
- Accepts Process ID, Arrival Time, Burst Time and Priority.
- Calculates scheduling performance metrics.
- Displays the process execution sequence using a Gantt chart.



Results – Process Input

- Entered process details.
- Process ID was provided.
- Arrival Time was provided.
- Burst Time was provided.
- Priority was provided.
- Scheduling type was selected.
- Process information was successfully accepted.

```
----- MAIN MENU -----  
1. Enter Process Details  
2. Display Process Details  
3. Non-Preemptive Priority Scheduling  
4. Preemptive Priority Scheduling  
5. Process Management Demo  
6. Anonymous Pipe Demo  
7. Named FIFO Demo  
8. Exit
```

```
-----  
Enter your choice: 1
```

```
===== PROCESS INPUT =====  
-----
```

```
Enter number of processes (1-20): 2
```

```
----- PROCESS 1 -----  
-----
```

```
PID          : 1  
Arrival Time : 2  
Burst Time   : 1  
Priority      : 3
```

```
----- PROCESS 2 -----  
-----
```

```
PID          : 2  
Arrival Time : 1  
Burst Time   : 4  
Priority      : 2
```

```
===== PROCESS DETAILS STORED SUCCESSFULLY =====  
-----
```

Results – Display Process Details

- Selected **Display Process Details**.
- Stored process information was displayed successfully.
- Process ID, Arrival Time, Burst Time and Priority were displayed.
- The entered process details were verified before scheduling.

```
----- MAIN MENU -----
1. Enter Process Details
2. Display Process Details
3. Non-Preemptive Priority Scheduling
4. Preemptive Priority Scheduling
5. Process Management Demo
6. Anonymous Pipe Demo
7. Named FIFO Demo
8. Exit
```

```
-----
Enter your choice: 2
```

```
=====
                        PROCESS INFORMATION
=====
```

PID	AT	BT	Priority
1	2	1	3
2	1	4	2

```
=====
```

```
Priority Rule:
Smaller priority number = Higher priority
```

Results – Non Preemptive

- Selected Non-Preemptive Priority Scheduling.
- Processes were considered according to priority.
- The selected process continued until completion.
- Process execution order was generated.
- Completion Time was calculated.
- Turnaround Time was calculated.
- Waiting Time and Response Time were calculated.

```
Enter your choice: 3

=====
NON-PREEMPTIVE PRIORITY SCHEDULING
=====

Priority Rule:
Smaller priority number = Higher priority

=====
SCHEDULING RESULTS
=====
PID    AT    BT    Priority  CT    TAT    WT    RT
-----
1       2     1     3         6     4      3     3
2       1     4     2         5     4      0     0
=====

Average Waiting Time   : 1.50
Average Turnaround Time : 4.00
Average Response Time  : 1.50

=====
GANTT CHART
=====

| IDLE | P2 | P1 |
0       1     5     6

=====
NON-PREEMPTIVE SCHEDULING COMPLETE
=====
```

Results – Preemptive

- Selected Preemptive Priority Scheduling.
- Processes were considered according to priority and arrival time.
- CPU execution can change when a higher-priority process becomes available.
- Process execution order was generated.
- Completion Time was calculated.
- Turnaround Time was calculated.
- Waiting Time and Response Time were calculated.

```
-----
Enter your choice: 4

=====
PREEMPTIVE PRIORITY SCHEDULING
=====

Priority Rule:
Smaller priority number = Higher priority

=====
SCHEDULING RESULTS
=====
PID    AT    BT    Priority CT    TAT    WT    RT
-----
1       2     1     3         6     4     3     3
2       1     4     2         5     4     0     0
=====

Average Waiting Time    : 1.50
Average Turnaround Time : 4.00
Average Response Time   : 1.50

=====
GANTT CHART
=====

| IDLE | P2 | P1 |
0      1      5      6

=====
PREEMPTIVE SCHEDULING COMPLETE
=====
```

Results – Process Management Demo

- Selected **Process Management Demo**.
- Demonstrates process creation and process control.
- `fork()` is used for process creation.
- Parent and child process execution is demonstrated.
- `wait()` is used for parent-child process synchronization.
- Process execution and termination are displayed in the terminal.

```
Enter your choice: 5

=====
PROCESS MANAGEMENT DEMO
=====

Parent Process ID : 639

Creating child process using fork()...

--- PARENT PROCESS ---
Parent Process ID : 639
Child Process ID : 662

Parent is waiting for child using wait()...

--- CHILD PROCESS ---
Child Process ID : 662
Parent Process ID : 639

Child is executing a new program using exec()...
total 64
-rwxrwxrwx 1 ketana_sindhu ketana_sindhu 30232 Sep 24 16:17 priority_scheduler
-rwxrwxrwx 1 ketana_sindhu ketana_sindhu 31721 Sep 24 09:51 priority_scheduler.c

Child process terminated normally.
Child Exit Status : 0

Parent process continues execution.

=====
PROCESS MANAGEMENT DEMO COMPLETE
=====
```

Results – Anonymous Pipe Demo

- Selected **Anonymous Pipe Demo**.
- Demonstrates communication between related processes.
- Data is transferred through an anonymous pipe.
- The process communication is displayed through terminal output.
- Successful data transfer between processes was verified.

Enter your choice: 6

ANONYMOUS PIPE DEMO

--- PARENT PROCESS ---

Parent PID : 639

Sending message through pipe...

Message sent successfully.

--- CHILD PROCESS ---

Child PID : 669

```
Message received: Hello from the Priority Scheduling parent process!
```

Child process completed.

ANONYMOUS PIPE DEMO COMPLETE

Results – Named FIFO Demo

- Selected **Named FIFO Demo**.
- Demonstrates inter-process communication using a named FIFO.
- A named communication channel is created.
- Data is transferred between processes through the FIFO.
- Successful communication is displayed in the terminal.

```
Enter your choice: 7
```

```
=====
                        NAMED FIFO DEMO
=====
```

```
--- PARENT PROCESS ---
```

```
Parent PID : 639
```

```
--- CHILD PROCESS ---
```

```
Child PID : 689
```

```
Waiting for message through FIFO...
```

```
Sending message through FIFO...
```

```
Message sent successfully.
```

```
Message received: Hello through Named FIFO from Priority Scheduling!
```

```
Child process completed.
```

```
=====
                        NAMED FIFO DEMO COMPLETE
=====
```

Results – Exit

- User selects **Exit**.
- The program terminates normally.
- Process management resources are released.
- The application returns control to the Linux terminal.
- Program execution ends successfully.

```
Enter your choice: 8
```

```
=====
                        SIGNAL DEMO
=====
```

```
--- PARENT PROCESS ---
--- CHILD PROCESS ---
Parent PID : 639
Child PID : 832
Child is waiting for SIGUSR1...
Sending SIGUSR1 to child...
SIGUSR1 sent successfully.
Child received SIGUSR1 successfully!
Child process resumed after receiving signal.
Child process completed.
Child process completed.
```

```
=====
                        SIGNAL DEMO COMPLETE
=====
```

HARDWARE & SOFTWARE

HARDWARE

- Computer / Laptop
- Keyboard
- Display

SOFTWARE

- Operating System: **Linux / Ubuntu / WSL2**
- Programming Language: **C**
- Compiler: **GCC**
- Interface: **Linux Terminal**
- Editor: **Visual Studio Code**
- System Programming: **Linux/POSIX APIs**
- Version Control: **Git / GitHub**

Conclusion

- Developed a **Priority Scheduling simulator** in a Linux environment.
- Implemented **Preemptive and Non-Preemptive Priority Scheduling**.
- Accepted Process ID, Arrival Time, Burst Time and Priority.
- Calculated Completion Time, Turnaround Time, Waiting Time and Response Time.
- Generated process execution order using a Gantt chart.
- Demonstrated CPU scheduling and process management concepts.
- Successfully tested the Priority Scheduling implementation.

Future Scope

Scalability

- Support a larger number of processes.
- Add additional CPU scheduling algorithms.
- Compare different scheduling algorithms.

User Experience

- Improve process input interface.
- Improve Gantt chart visualization.
- Provide clearer scheduling results.

Extension

- Develop a graphical user interface.
- Add scheduling performance visualization.
- Extend the simulator with additional Linux/POSIX features.

Thank you

Priority Scheduling — Team 20 | Section - 6

Somisetty Ketana Sindhu-2520030487

Yerrolaa Eekshitha— 2520030615

Poojitha Rani— 2520030460
